ROAD VEHICLE BREAKDOWN ASSISTANCE

Submitted in partial fulfilment of the requirements
of the degree of

BACHELOR OF ENGINEERING

in

INFORMATION TECHNOLOGY

(A.Y. 2023-24)

by

Ritesh Purohit (Roll No. 04)
Swapnil Patil (Roll No. 37)
Vishwa Rajput (Roll No. 49)

Subject: INDUSTRY PRACTICES

Formative Assessment – II (FA-II)

Choice Based Credit Grading System with Holistic Student Development

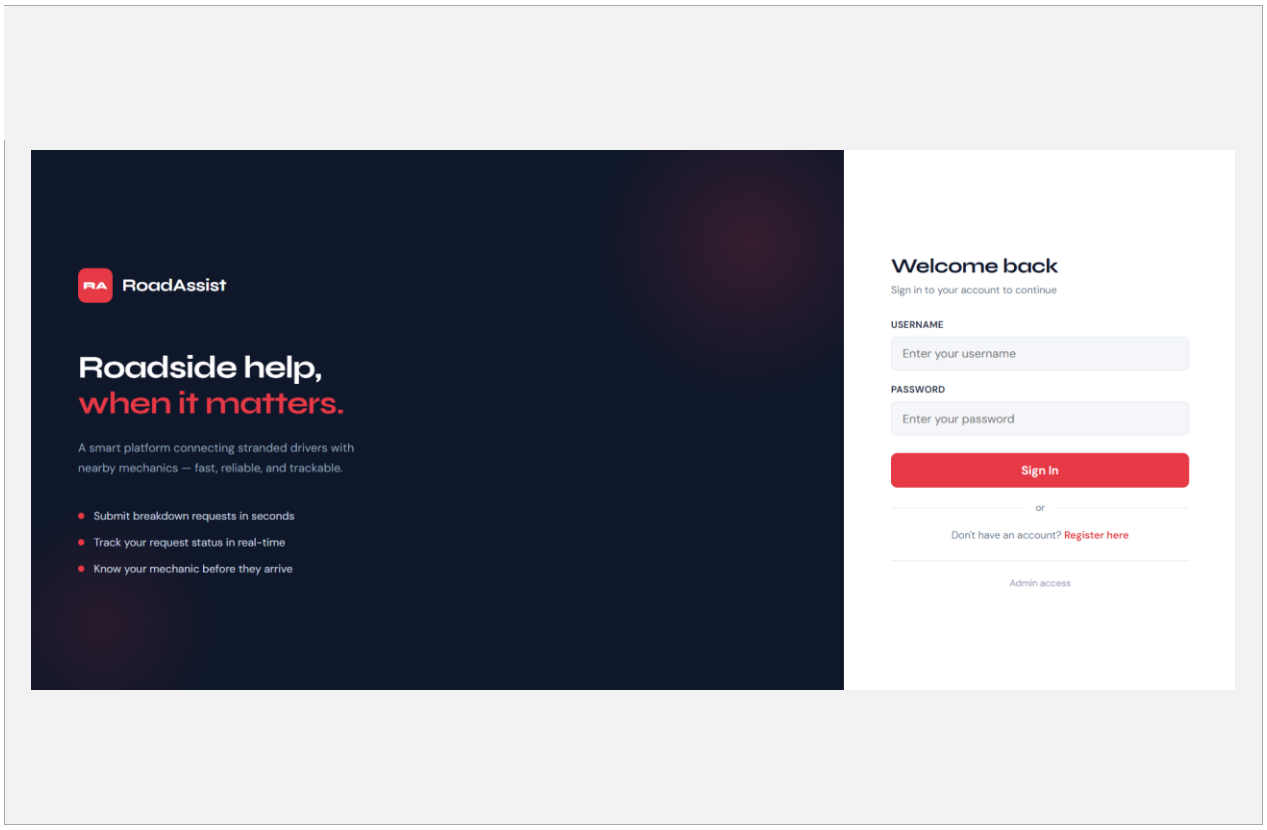
Results and Discussions

The Road Vehicle Breakdown Assistance System was successfully implemented and deployed as a fully functional web application accessible at https://roadassist.onrender.com. The system underwent comprehensive functional testing across all modules, and the results demonstrate that all core features operate as intended. The following subsections present the key results along with illustrative screenshots of the deployed application.

1. Login and Registration Page

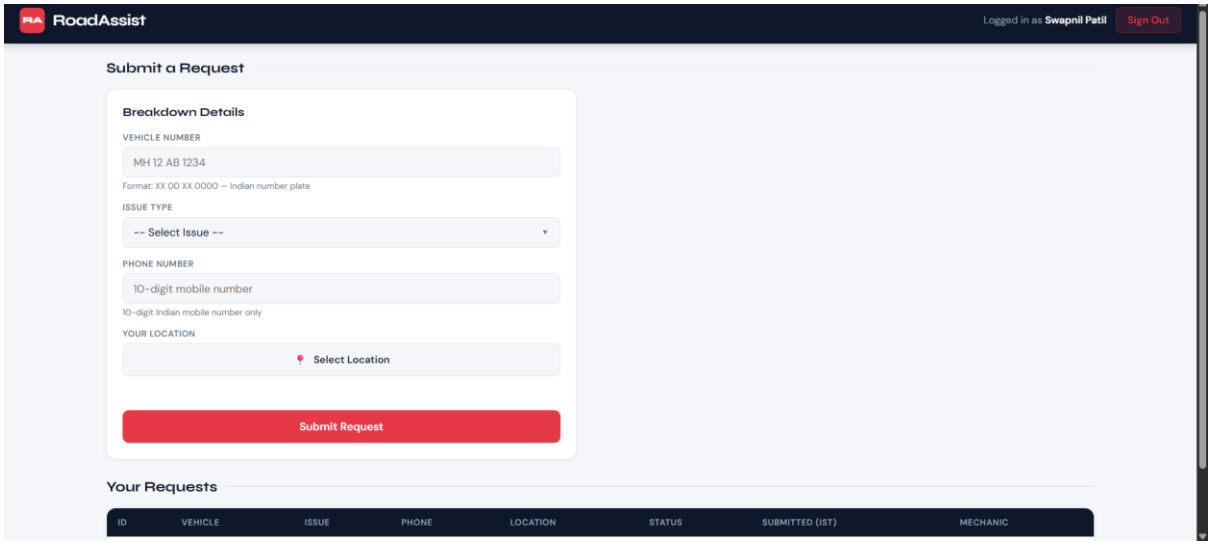
The login and registration interface presents a professional split-screen layout. The left panel displays the RoadAssist branding, while the right panel provides the authentication form. New users can register by providing a username, phone number, and password. Existing users are authenticated and redirected to their respective dashboards based on their assigned role

(CUSTOMER or ADMIN). Input validation is enforced on all fields, including phone number format and password strength.

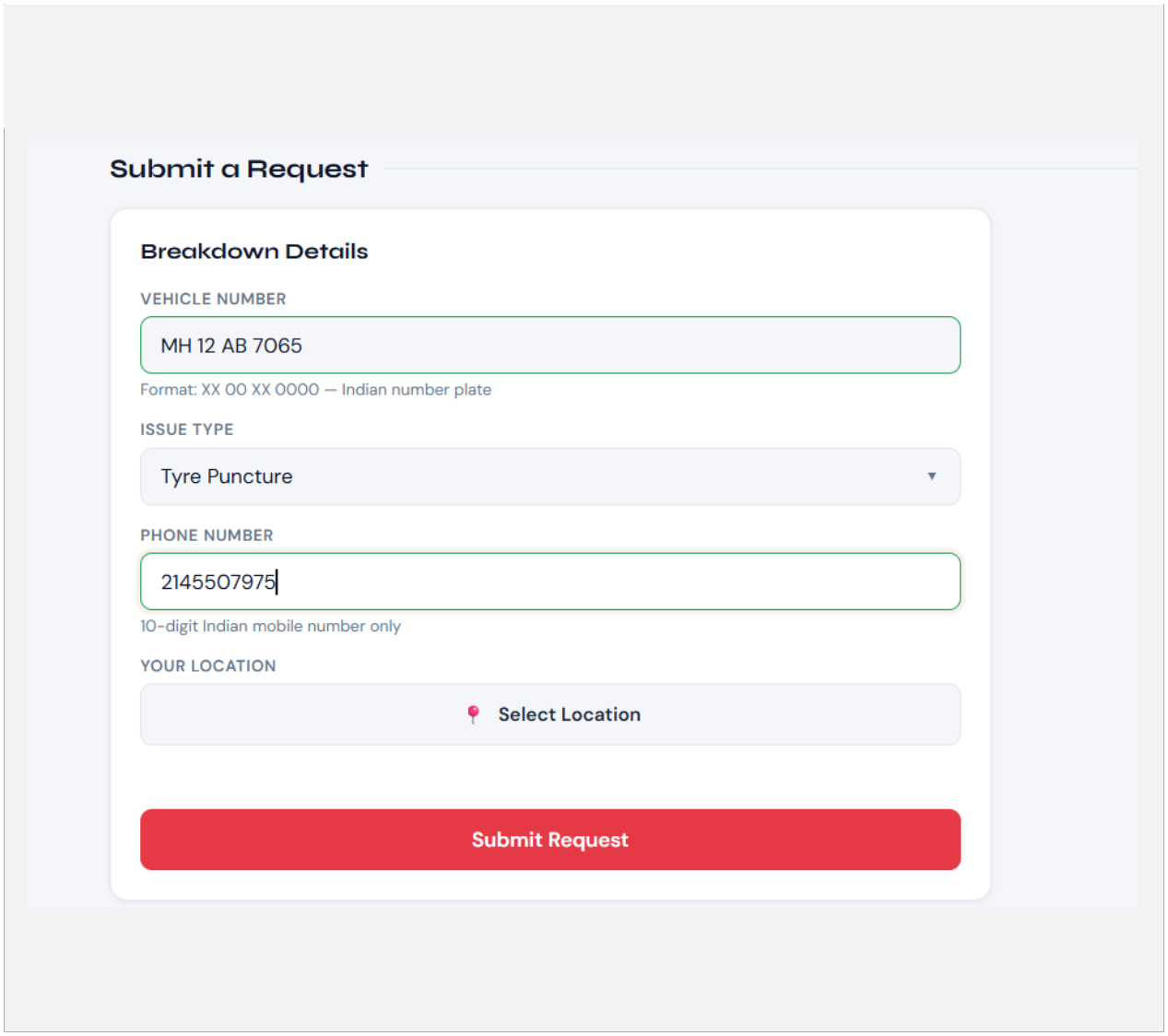


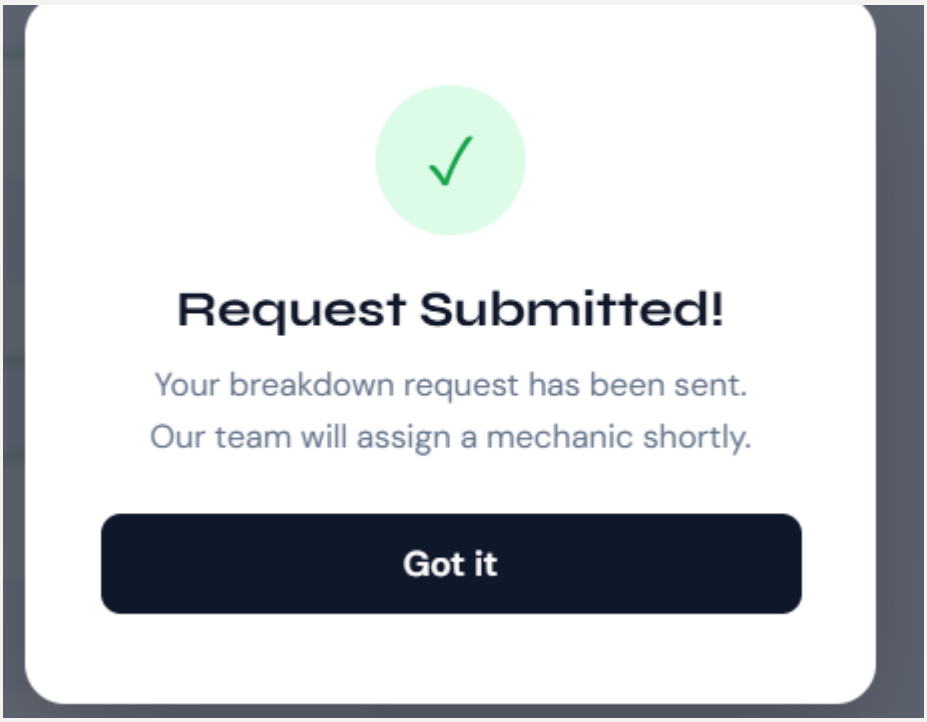
2. Customer Dashboard

Upon successful login, the customer is presented with a dashboard displaying all previously submitted breakdown requests along with their current status (PENDING, ACCEPTED, or



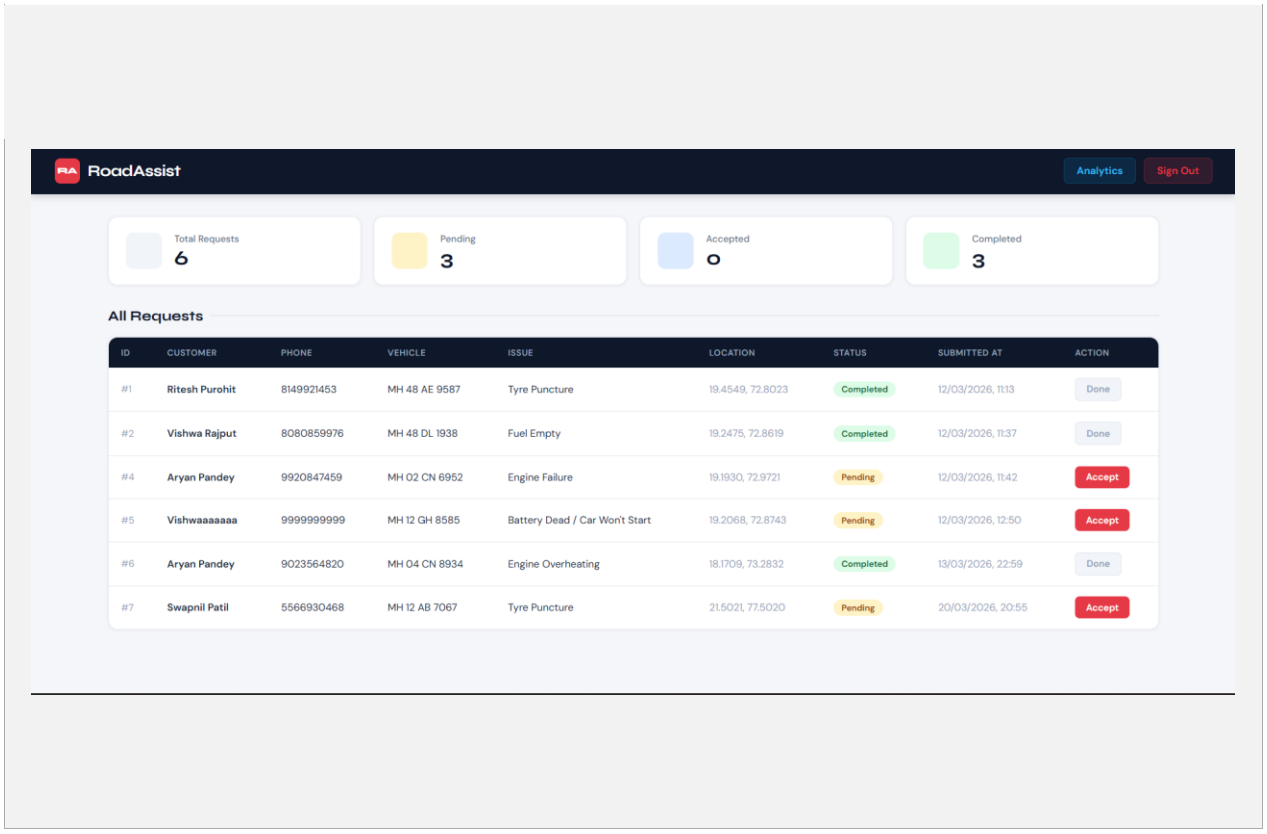
COMPLETED). The breakdown submission form enables the customer to enter vehicle registration details in standard Indian format, select the breakdown category, provide a contact number, and capture their location either via live GPS or interactive map selection. The map picker enforces India-only location selection through coordinate boundary validation.


Submit a Request
Breakdown Details
VEHICLE NUMBER
MH 12 AB 7065
Format: XX 00 XX 0000 — Indian number plate
ISSUE TYPE
Tyre Puncture
PHONE NUMBER
2145507975
10-digit Indian mobile number only
YOUR LOCATION
Select Location
Submit Request



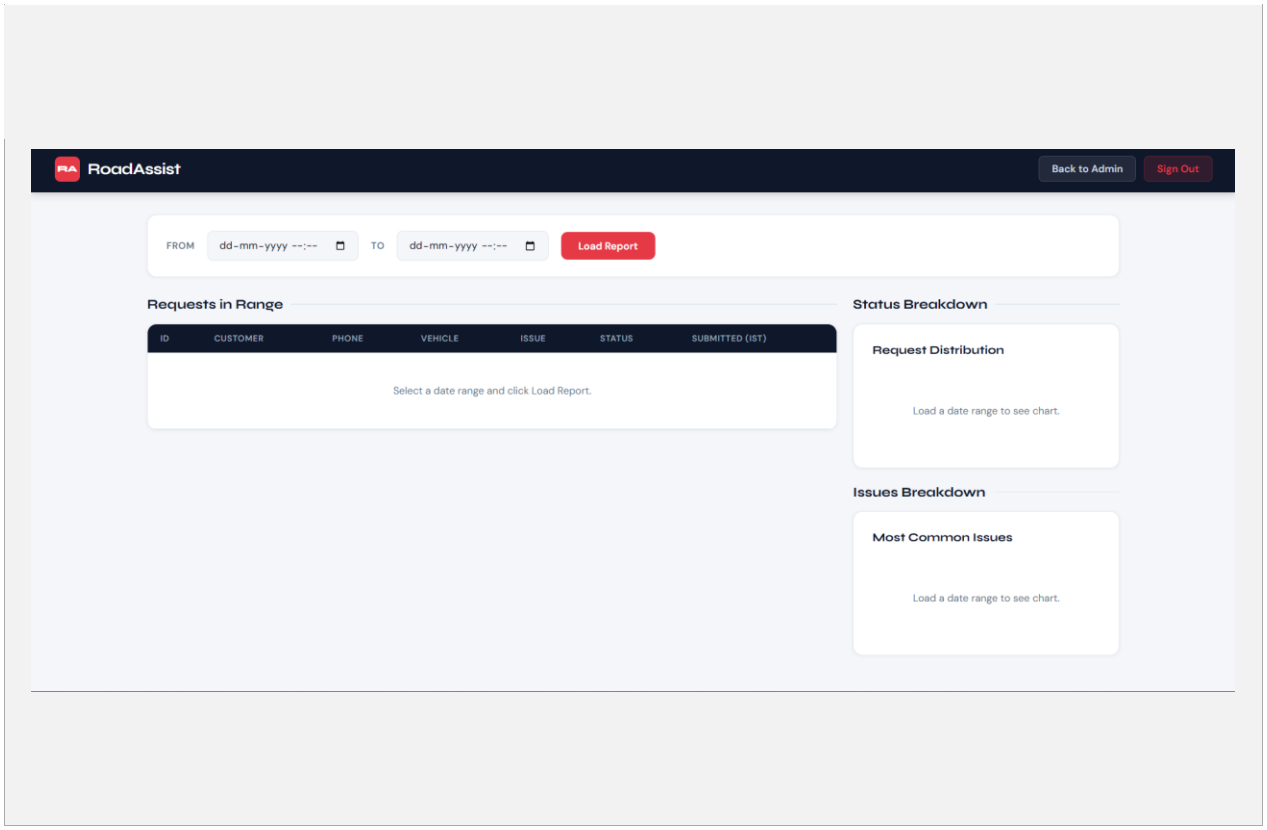
3. Admin Dashboard

The administrative dashboard provides the service coordinator with a consolidated view of all breakdown requests received across the platform. Summary statistics cards at the top of the dashboard display the total count of requests categorised by status. The coordinator can accept a pending request by assigning a mechanic name, contact number, estimated arrival time, and mechanic location coordinates. Upon acceptance, the request status transitions to ACCEPTED and the assigned mechanic's details become visible on the customer dashboard.



4. Analytics Dashboard

The analytics module allows the administrator to filter breakdown requests within a specified date and time range. The filtered results are presented in a tabular format with IST timestamps. Two interactive doughnut charts provide visual insights: one illustrating the distribution of requests by resolution status, and another depicting the frequency of the ten most commonly reported breakdown issue types. A PDF report encompassing the summary statistics and complete request data can be generated and downloaded directly from this page.



5. Discussion of Results

The system successfully achieves all primary objectives outlined during the design phase. The end-to-end workflow — from customer registration and breakdown submission to mechanic assignment and resolution — functions reliably across desktop and mobile browsers. The India-restricted location selection feature ensures geographical relevance of the platform, while the IST timestamp correction guarantees accurate temporal records for all transactions.

The deployment on Render and Clever Cloud demonstrates that a fully functional, cloud-hosted web application can be maintained at zero cost within the constraints of free-tier infrastructure. The application exhibits acceptable response times under typical load conditions, with an initial wake-up latency of approximately 50 seconds attributable to the free-tier spin-down behaviour of the Render hosting service.

Conclusion and Future Scope

Conclusion

The Road Vehicle Breakdown Assistance System has been successfully designed, developed, and deployed as a comprehensive web-based platform addressing the practical challenges associated with roadside vehicle breakdown management. The project demonstrates the effective integration of Java Spring Boot, MySQL, Leaflet.js, Chart.js, and Docker within a layered application architecture to deliver a reliable, scalable, and user-centric solution.

The system fulfils its core mandate by providing motorists with a structured digital mechanism to register breakdown incidents and share precise location data, while empowering service coordinators with the tools necessary to manage request assignment and monitor resolution progress. The analytics module further extends the platform's utility by enabling data-driven operational insights through interactive visualisation and exportable PDF reporting.

From an academic perspective, this project has provided the development team with valuable hands-on experience across the full spectrum of software engineering activities, encompassing requirements analysis, system design, frontend and backend development, database modelling, API design, containerisation, and cloud deployment. The successful delivery of a live, production-grade application within the scope of the Industry Practices subject reflects the team's technical competence and collaborative discipline.

Future Scope

The following enhancements are identified as priority improvements for subsequent iterations of the platform:

1. SMS and email notification integration upon mechanic assignment, utilising communication platforms such as Twilio or SendGrid, to ensure customers receive timely alerts independent of active dashboard sessions.
2. Introduction of dedicated mechanic accounts with independent login credentials, replacing the current administrator-mediated assignment model and enabling mechanics to self-manage their availability and service history.

3. Real-time mechanic location tracking on the customer map using WebSocket-based communication, enabling dynamic monitoring of the mechanic's approach and improving customer confidence.
4. Implementation of a post-service rating and review system to support quality assurance, accountability, and informed service provider selection.
5. Multi-language support, with priority given to Hindi and Marathi, to enhance platform accessibility for non-English-speaking users across India.
6. Development of native Android and iOS mobile applications to extend the platform beyond web browsers and leverage device-native capabilities such as push notifications and background location services.
7. Integration of a secure payment gateway to facilitate digital settlement of service charges within the platform.
8. Enhancement of the authentication subsystem with BCrypt password hashing and JWT-based stateless session management to meet industry security standards.

References

- Spring Boot Official Documentation. Retrieved from: https://spring.io/projects/spring-boot
- MySQL 8.0 Reference Manual. Oracle Corporation. Retrieved from: https://dev.mysql.com/doc/refman/8.0/en/
- Leaflet.js Interactive Map Library Documentation. Retrieved from: https://leafletjs.com/reference.html
- Chart.js Data Visualisation Library Documentation. Retrieved from: https://www.chartjs.org/docs/latest/
- jsPDF – Client-side PDF Generation Documentation. Retrieved from: https://rawgit.com/MrRio/jsPDF/master/docs/
- Render Cloud Deployment Documentation. Retrieved from: https://render.com/docs
- Clever Cloud MySQL Add-on Documentation. Retrieved from: https://www.clever-cloud.com/doc/addons/mysql/
- Docker Official Documentation. Retrieved from: https://docs.docker.com/
- OpenStreetMap Contributors. Tile data available under ODbL. Retrieved from: https://www.openstreetmap.org/
- Baeldung. Spring Boot REST API Tutorial. Retrieved from: https://www.baeldung.com/spring-boot-start

Code

This section presents the key source code components of the Road Vehicle Breakdown Assistance System. The complete codebase is maintained on GitHub at: https://github.com/Ritesh-453/roadassist

1. RoadassistApplication.java – Main Entry Point

```
package com.example.roadassist;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RoadassistApplication {
    public static void main(String[] args) {
        SpringApplication.run(RoadassistApplication.class, args);
    }
}
```

2. AppUser.java – User Entity

```
@Entity
@Table(name = "app_user")
public class AppUser {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String username;
    private String password;
    private String role;      // CUSTOMER or ADMIN
    private String phone;
    // Getters and Setters
}
```

3. ServiceRequest.java – Request Entity

```
@Entity
public class ServiceRequest {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String customerName;
    private String username;
    private String vehicleNumber;
    private String issue;
```

```
    private String phone;
    private Double latitude;
    private Double longitude;
    private String status;    // PENDING / ACCEPTED / COMPLETED
    private LocalDateTime createdAt;
    private String mechanicName;
    private String mechanicPhone;
    private String estimatedArrival;
    private String mechanicLatitude;
    private String mechanicLongitude;
    // Getters and Setters
}
```

4. AuthController.java – Login and Register APIs

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @PostMapping("/register")
    public ResponseEntity<?> register(@RequestBody AppUser user) {
        user.setRole("CUSTOMER");
        userRepository.save(user);
        return ResponseEntity.ok("User registered");
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody AppUser user) {
        AppUser found = userRepository.findByUsername(user.getUsername());
        if (found != null && found.getPassword().equals(user.getPassword())) {
            return ResponseEntity.ok(Map.of("role", found.getRole()));
        }
        return ResponseEntity.status(401).body("Invalid credentials");
    }
}
```

5. ServiceRequestController.java – Request Management APIs

```
@RestController
@RequestMapping("/api/requests")
public class ServiceRequestController {

    @PostMapping
    public ServiceRequest create(@RequestBody ServiceRequest req) {
```

```
        req.setStatus("PENDING");
        req.setCreatedAt(LocalDateTime.now());
        return repository.save(req);
    }

    @PatchMapping("/{id}/accept")
    public ServiceRequest accept(@PathVariable Long id,
                                 @RequestBody ServiceRequest details) {
        ServiceRequest req = repository.findById(id).orElseThrow();
        req.setStatus("ACCEPTED");
        req.setMechanicName(details.getMechanicName());
        req.setMechanicPhone(details.getMechanicPhone());
        req.setEstimatedArrival(details.getEstimatedArrival());
        req.setMechanicLatitude(details.getMechanicLatitude());
        req.setMechanicLongitude(details.getMechanicLongitude());
        return repository.save(req);
    }
}
```

6. IST Timestamp Function – Frontend JavaScript

```
function formatIST(dtStr) {
  // Append Z to treat server string as UTC, then add 330 min for IST
  const utc = new Date(dtStr + 'Z');
  const ist = new Date(utc.getTime() + 330 * 60000);
  const dd   = String(ist.getUTCDate()).padStart(2, '0');
  const mm   = String(ist.getUTCMonth() + 1).padStart(2, '0');
  const yyyy = ist.getUTCFullYear();
  const hh   = String(ist.getUTCHours()).padStart(2, '0');
  const min  = String(ist.getUTCMinutes()).padStart(2, '0');
  return dd + '/' + mm + '/' + yyyy + ', ' + hh + ':' + min;
}
```

7. application.properties – Production Configuration

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
server.port=${PORT:8082}
```

8. Dockerfile – Container Configuration

```
FROM maven:3.9.6-eclipse-temurin-17 AS build
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8082
ENTRYPOINT ["java", "-jar", "app.jar"]
```

— End of FA-II Report —